

关系模型应用数学方法来处理数据库中的数据。1962年 CODASYL 发表的“信息代数”最早提出将这类方法用于数据处理,1968年 David Child 在 IBM 7090 机上实现了集合论数据结构,但系统、严格地提出关系模型的是 Edgar F. Codd。

1970年,Edgar F. Codd 在美国计算机学会(ACM)会刊上发表了题为 *A Relational Model of Data for Large Shared Data Banks* 的论文,开创了数据库系统的新纪元。1983年,该论文被 ACM 列为 1958 年以来的 1/4 世纪中具有里程碑意义的 25 篇研究论文之一。此后,Edgar F. Codd 又陆续发表了多篇论文,奠定了关系数据库的理论基础。

20 世纪 70 年代末,关系方法的理论研究和软件系统的研制紧密结合,均取得了丰硕的成果。例如,IBM 公司的 San Jose 实验室在 IBM 370 系列机上研制的关系数据库实验系统 System R 历时 6 年获得成功。1981 年,IBM 公司宣布具有 System R 全部特征的新的数据库软件产品 SQL/DS 问世。

与 System R 同期,1973 年美国加州大学伯克利分校的 Michael Stonebraker 和 Eugene Wong 研制了 INGRES 关系数据库实验系统,并由 INGRES 公司发展成为 INGRES 数据库产品。在 INGRES 的基础上,20 世纪 80 年代中期 PostgreSQL 系统发布并成功开放其源代码。

在此期间 Oracle 公司成立,1978 年发布了 Oracle 1.0,以后短短十几年中 Oracle 产品不断成熟,Oracle 公司也逐渐发展成为著名的数据库软件及服务供应商。

此后数据库领域又涌现出许多优秀的关系数据库产品,关系数据库系统逐渐从实验室走入社会,得到了广泛的应用和快速的发展。

本章导读

本章基于第一章对关系模型及其基本术语的介绍,进一步深入讲解关系模型的有关内容。

本章按照数据模型的三要素,介绍关系模型的数据结构及形式化定义、关系操作和关系数据语言的大致分类、关系的三类完整性约束。希望读者能牢固掌握关系模型三个组成部分的内涵,为学习 SQL 和数据库安全性与完整性打下基础。

本章还讲解了关系代数和关系演算。关系代数用对关系的运算来表达查询要求,关系代数

表达式是关系数据库管理系统查询优化中代数优化的依据,读者应掌握关系代数表达查询的方法。关系演算以数理逻辑中的谓词演算为基础,用谓词来表达查询要求。本章讲解了元组关系演算语言 ALPHA 和域关系演算语言 QBE,希望读者基本掌握关系演算语言的概念和使用方法,并通过习题加强练习。

2.1 关系模型的数据结构及形式化定义

本书第1章1.2.6小节非形式化地介绍了关系模型的基本概念。由于关系模型是建立在集合代数基础上的,本节从集合论角度给出关系数据结构的形式化定义,以及关系模式的形式化表示,并简要介绍关系数据库和关系模型的存储结构。

2.1.1 关系

关系模型的数据结构非常简单,只包含单一的数据结构——关系。在用户看来,关系模型中数据的逻辑结构是一张二维表。

关系模型的数据结构虽然简单却能够表达丰富的语义,描述现实世界的实体以及实体间的各种联系。也就是说,在关系模型中,现实世界的实体以及实体间的各种联系均用单一的结构类型,即关系来表示。

1. 域

定义 2.1 域是一组具有相同数据类型的值的集合。

例如,自然数、整数、实数、长度小于 25 B 的变长字符串集合、 $\{0,1\}$ 、 $\{\text{男},\text{女}\}$ 、 $0\sim 100$ 之间的正整数等,都可以是域。

2. 笛卡儿积

笛卡儿积(Cartesian product)是域上的一种集合运算。

定义 2.2 给定一组域 $D_1, D_2, \dots, D_n$, 允许其中某些域是相同的, $D_1, D_2, \dots, D_n$ 的笛卡儿积为

$$D_1 \times D_2 \times \dots \times D_n = \{(d_1, d_2, \dots, d_n) \mid d_i \in D_i, i=1, 2, \dots, n\}$$

其中,每一个元素 $(d_1, d_2, \dots, d_n)$ 叫作一个 n 元组(n -tuple),简称元组。元组中的每一个值 d_i 叫作一个分量。

一个域允许的不同取值个数称为这个域的基数(cardinal number)。

若 $D_i (i=1, 2, \dots, n)$ 为有限集,其基数为 $m_i (i=1, 2, \dots, n)$, 则 $D_1 \times D_2 \times \dots \times D_n$ 的基数 M 为

$$M = \prod_{i=1}^n m_i$$

笛卡儿积可表示为一张二维表。表中的每一行对应一个元组,表中的每一列的值来自一个域。例如,给出三个域:

$$D_1 = \text{导师 (SUPERVISOR)} = \{\text{张清玫, 刘逸}\}$$

$$D_2 = \text{专业 (MAJOR)} = \{\text{计算机科学与技术, 信息管理与信息系统}\}$$

$$D_3 = \text{研究生 (POSTGRADUATE)} = \{\text{李勇, 刘晨, 王敏}\}$$

则 D_1 、 D_2 、 D_3 的笛卡儿积如下:

$$D_1 \times D_2 \times D_3 = \{(\text{张清玫, 计算机科学与技术, 李勇}), (\text{张清玫, 计算机科学与技术, 刘晨}),$$

$$(\text{张清玫, 计算机科学与技术, 王敏}), (\text{张清玫, 信息管理与信息系统, 李勇}),$$

$$(\text{张清玫, 信息管理与信息系统, 刘晨}), (\text{张清玫, 信息管理与信息系统, 王敏}),$$

$$(\text{刘逸, 计算机科学与技术, 李勇}), (\text{刘逸, 计算机科学与技术, 刘晨}),$$

$$(\text{刘逸, 计算机科学与技术, 王敏}), (\text{刘逸, 信息管理与信息系统, 李勇}),$$

$$(\text{刘逸, 信息管理与信息系统, 刘晨}), (\text{刘逸, 信息管理与信息系统, 王敏})\}$$

其中,“(张清玫, 计算机科学与技术, 李勇)”“(张清玫, 计算机科学与技术, 刘晨)”等都是元组,“张清玫”“计算机科学与技术”“李勇”“刘晨”等都是元组的分量。

该笛卡儿积的基数为 $2 \times 2 \times 3 = 12$ 。也就是说, $D_1 \times D_2 \times D_3$ 共有 $2 \times 2 \times 3 = 12$ 个元组。这 12 个元组可列成一张二维表, 如表 2.1 所示。

表 2.1 笛卡儿积示例

导师 SUPERVISOR	专业 MAJOR	研究生 POSTGRADUATE
张清玫	计算机科学与技术	李勇
张清玫	计算机科学与技术	刘晨
张清玫	计算机科学与技术	王敏
张清玫	信息管理与信息系统	李勇
张清玫	信息管理与信息系统	刘晨
张清玫	信息管理与信息系统	王敏
刘逸	计算机科学与技术	李勇
刘逸	计算机科学与技术	刘晨
刘逸	计算机科学与技术	王敏
刘逸	信息管理与信息系统	李勇
刘逸	信息管理与信息系统	刘晨
刘逸	信息管理与信息系统	王敏

3. 关系

一般来说, 在关系模型中 $D_1, D_2, \dots, D_n$ 的笛卡儿积是没有实际语义的, 只有它的某个真子集才有实际含义。

例如,可以发现表 2.1 的笛卡儿积中许多元组是没有意义的。因为在学校中一个专业方向有多个导师,而一个导师只在一个专业方向带研究生;一个导师可以带多个研究生,而一个研究生只有一个导师,学习某一个专业。因此,表 2.1 中的一个子集才是有意义的,可以表示导师与研究生的关系。把该关系取名为 SMP,如表 2.2 所示,李勇和刘晨是计算机科学与技术专业张清玫老师的研究生,王敏是信息管理与信息系统专业刘逸老师的研究生。

表 2.2 导师-研究生关系 SMP

导师 SUPERVISOR	专业 MAJOR	研究生 POSTGRADUATE
张清玫	计算机科学与技术	李勇
张清玫	计算机科学与技术	刘晨
刘逸	信息管理与信息系统	王敏

关系 SMP 包含的属性为 SUPERVISOR、MAJOR 和 POSTGRADUATE,则其关系模式可以表示为

SMP(SUPERVISOR, MAJOR, POSTGRADUATE)

下面给出关系及关系模式的形式化定义和相关概念。

定义 2.3 给定一组域 $D_1, D_2, \dots, D_n$, 允许其中某些域是相同的, $D_1, D_2, \dots, D_n$ 的笛卡儿积 $D_1 \times D_2 \times \dots \times D_n$ 的子集称为这组域上的关系, 表示为

$$R(D_1, D_2, \dots, D_n)$$

这里 R 表示关系名, n 是关系的目或度(degree)。

当 $n=1$ 时, 称该关系为一元关系(unary relation)或单元关系、单目关系。

当 $n=2$ 时, 称该关系为二元关系(binary relation)或二目关系。

关系中的每个元素是关系中的元组, 通常用 t 表示。

关系是笛卡儿积的有限子集, 所以关系是一张二维表, 表的每一行对应一个元组, 表的每一列对应一个域。由于域可以相同, 为了加以区分, 必须对每一列起一个名字, 称为属性。 n 目关系必有 n 个属性。

例如, SMP(SUPERVISOR, MAJOR, POSTGRADUATE)有三个属性, 是一个三目关系。

(张清玫, 计算机科学与技术, 李勇)、(张清玫, 计算机科学与技术, 刘晨)和(刘逸, 信息管理与信息系统, 王敏)是 SMP 关系的三个元组。

关系可以有三种类型: 基本关系(通常又称为基本表或基表)、查询结果和视图。其中, 基本关系是实际存在的表, 它是实际存储数据的逻辑表示; 查询结果是查询执行产生的结果对应的临时表; 视图是由基本表或其他视图导出的虚表, 不存储实际数据。

按照定义 2.2, 关系可以是一个无限集合。例如, 某个域是整数, 而整数是个无限集合。此外, 在数学上组成笛卡儿积的域不满足交换律, 所以按照数学定义, $(d_1, d_2, \dots, d_n) \neq (d_2,$

$d_1, \dots, d_n$)。因此, 当关系作为数据模型的数据结构时, 需要给予如下的限定和扩充。

① 无限关系在数据库系统中是无意义的。因此, 限定关系模型中的关系必须是有限集合。

② 通过为关系的每个列附加一个属性名的方法取消关系属性的有序性, 使得列的次序可以任意交换:

$$(d_1, d_2, \dots, d_i, d_j, \dots, d_n) = (d_1, d_2, \dots, d_j, d_i, \dots, d_n) \quad (i, j = 1, 2, \dots, n)$$

因此, 基本关系具有以下 6 条性质。

① 列是同质的(homogeneous), 即每一列中的分量是同一类型的数据, 来自同一个域。

② 不同的列可出自同一个域, 称其中的每一列为一个属性, 不同的属性要给予不同的属性名。例如, 在上面的例子中, 也可以只给出两个域:

人(PERSON) = {张清玫, 刘逸, 李勇, 刘晨, 王敏}

专业(MAJOR) = {计算机科学与技术, 信息管理与信息系统}

SMP 关系的导师属性和研究生属性都是从 PERSON 域中取值。为了避免混淆, 必须给这两个属性取不同的属性名, 而不能直接使用域名。因此, 定义导师属性名为 SUPERVISOR, 研究生属性名为 POSTGRADUATE。

③ 列的顺序无所谓, 即列的次序可以任意交换。由于列顺序是无关紧要的, 因此在许多实际的关系数据库产品中增加新属性时, 永远是将其插至最后一列。

④ 任意两个元组的码不能取相同的值。

⑤ 行的顺序无所谓, 即行的次序可以任意交换。

⑥ 分量必须取原子值, 即每一个分量都必须是不可分的数据项。

关系模型要求关系必须满足一定的规范条件, 其中最基本的一条就是元组的每一个分量必须是一个不可分的数据项。仍以导师-研究生关系为例, 表 2.3 虽然很好地表达了两之间的一对多联系, 即一个导师可以指导多个研究生, 但是由于属性“POSTGRADUATE”中分量取了两个值“PG1”“PG2”, 不符合规范化的要求, 因此这样的关系在关系数据库中是不允许的。直观地描述, 表 2.3 中还有一个小表, 不符合规范化的条件。

表 2.3 非规范化关系

SUPERVISOR	MAJOR	POSTGRADUATE	
		PG1	PG2
张清玫	计算机科学与技术	李勇	刘晨
刘逸	信息管理与信息系统	王敏	

小表

规范化的关系简称为范式(normal form, NF), 将在第 6 章关系数据理论中做进一步讲解。

2.1.2 关系模式

第 1 章 1.3.1 小节已经简单介绍了数据库系统中模式的概念。强调了在数据库中要区分数

据模型的型和值。在关系模型中,关系是元组的集合,关系是值;关系模式是对关系的描述,关系模式是型。

那么关系模式需要描述关系的哪些方面呢?

首先,关系模式必须描述关系元组集合的结构,即它由哪些属性构成,这些属性来自哪些域,以及属性与域之间的映像关系。

其次,关系模式要描述关系的完整性约束。现实世界随着时间在不断地变化,因而在不同的时刻关系模式所描述的关系也会不断变化。例如,在“学生”关系中,2020年某校的在校生与2019年就不同。但是,现实世界的许多已有事实和规则限定了关系模式所有可能的关系必须满足一定的完整性约束条件。这些约束条件或者通过对属性取值范围进行限定来描述,或者通过属性与属性之间的相互关联和相互约束反映出来。

定义 2.4 关系的描述称为**关系模式**(relation schema)。它可以形式化地表示为

$$R(U, D, \text{DOM}, F)$$

其中 R 为关系名, U 为组成该关系的属性的属性名集合, D 为 U 中属性所来自的域, DOM 为属性向域的映像集合, F 为属性间数据依赖关系的集合。

属性间的数据依赖将在第6章讨论,本章中关系模式仅涉及关系名、属性名、域名、属性向域的映像4部分,即 $R(U, D, \text{DOM})$ 。

例如,在上面例子中,由于导师和研究生出自同一个域——人:

$$\text{DOM}(\text{SUPERVISOR}) = \text{DOM}(\text{POSTGRADUATE}) = \text{PERSON}$$

所以要取不同的属性名,并在模式中定义属性向域的映像,即说明它们分别出自哪个域。

关系模式通常简记为

$$R(U)$$

或

$$R(A_1, A_2, \dots, A_n)$$

其中 R 为关系名, $A_1, A_2, \dots, A_n$ 为属性名;而域名及属性向域的映像则常直接说明为属性的类型、长度。

若关系模式中的某一个属性或一组属性的值能唯一地标识一个元组,而它的真子集不能唯一地标识一个元组,则称该属性或属性组为**候选码**(candidate key)。

若一个关系有多个候选码,则选定其中一个为**主码**(primary key),或称**主键**。

例如,在导师-研究生关系 $\text{SMP}(\text{SUPERVISOR}, \text{MAJOR}, \text{POSTGRADUATE})$ 中,假设研究生不会重名(这在实际生活中可能做不到,这里只是为了举例方便),则 POSTGRADUATE 属性的每一个值都唯一地标识了一个元组,因此可以作为 SMP 关系的主码,用在其名称下加下划线表示。

候选码的诸属性称为主属性(prime attribute)。不包含在任何候选码中的属性称为非主属性(non prime attribute)或非码属性(non-key attribute)。

在最简单的情况下, 候选码只包含一个属性。在最极端的情况下, 关系模式的所有属性是这个关系模式的候选码, 称为全码(all-key)。

关系是关系模式在某一时刻的状态或内容。关系模式是静态的、稳定的, 而关系是动态的、随时间不断变化的, 这是因为关系操作在不断地更新着数据库中的数据。例如, “学生”关系模式在不同的学年是相同的, 而“学生”关系是不同的。在实际工作中, 人们常常把关系模式和关系都笼统地称为关系, 这可以根据上下文加以区别, 希望读者注意。

2.1.3 关系数据库

支持关系模型的数据库系统称为关系数据库系统。在关系模型中, 实体以及实体间的联系都是用关系来表示的。例如, “学生”实体、“课程”实体、学生与课程之间选修课程的多对多联系都可以分别用一个关系模式来描述。

“学生”关系模式: Student(Sno, Sname, Ssex, Sbirthdate, Smajor), 包括学号、姓名、性别、出生日期和主修专业等属性。

“课程”关系模式: Course(Cno, Cname, Ccredit, Cpno), 包括课程号、课程名、学分、先修课(直接先修课)等属性。

“学生选课”关系模式: SC(Sno, Cno, Grade, Semester, Teachingclass), 包括学号、课程号、成绩、开课学期、教学班等属性。

在一个关系数据库中, 某一时刻所有关系模式对应的关系的集合构成一个关系数据库。例如图 2.1 就是一个“学生选课”数据库示例, 该数据库包含 3 个关系。

关系数据库也有类型和值之分。关系数据库的类型就是关系数据库中所有关系模式的集合, 是对关系数据库的描述, 通常称为关系数据库模式。关系数据库的值是这些关系模式在某一时刻对应的关系的集合, 通常称为关系数据库。

2.1.4 关系模型的存储结构

关系模型是关系数据的逻辑结构, 用关系数据定义语言描述, 例如第 3 章将介绍的关系数据库标准语言 SQL。

支持关系模型的关系数据库管理系统(relational database management system, RDBMS)将以一定的组织方式存储和管理数据, 即设计和实现关系模型的存储结构, 这是关系数据库管理系统的重要职责之一。

例如, 有的关系数据库管理系统中一个表对应一个操作系统文件, 将物理数据组织的许多任务交给操作系统完成; 有的关系数据库管理系统则从操作系统那里申请若干个大的文件, 自己划分文件空间, 组织表、索引等存储结构并进行存储管理。

本书第 9 章将专门讲解关系数据库存储管理, 重点介绍基于磁盘的数据库的组织与存储。

Student

学号 Sno	姓名 Sname	性别 Ssex	出生日期 Sbirthdate	主修专业 Smajor
20180001	李勇	男	2000-3-8	信息安全
20180002	刘晨	女	1999-9-1	计算机科学与技术
20180003	王敏	女	2001-8-1	计算机科学与技术
20180004	张立	男	2000-1-8	计算机科学与技术
20180005	陈新奇	男	2001-11-1	信息管理与信息系统
20180006	赵明	男	2000-6-12	数据科学与大数据技术
20180007	王佳佳	女	2001-12-7	数据科学与大数据技术

(a)

Course

课程号 Cno	课程名 Cname	学分 Ccredit	先修课 Cpno
81001	程序设计基础与C语言	4	
81002	数据结构	4	81001
81003	数据库系统概论	4	81002
81004	信息系统概论	4	81003
81005	操作系统	4	81001
81006	Python语言	3	81002
81007	离散数学	4	
81008	大数据技术概论	4	81003

(b)

SC

学号 Sno	课程号 Cno	成绩 Grade	开课学期 Semester	教学班 Teachingclass
20180001	81001	85	20192	81001-01
20180001	81002	96	20201	81002-01
20180001	81003	87	20202	81003-01
20180002	81001	80	20192	81001-02
20180002	81002	98	20201	81002-01
20180002	81003	71	20202	81003-02
20180003	81001	81	20192	81001-01
20180003	81002	76	20201	81002-02
20180004	81001	56	20192	81001-02
20180004	81002	97	20201	81002-02
20180005	81003	68	20202	81003-01

(c)

图 2.1 “学生选课”数据库示例

2.2 关系操作

关系模型给出了关系操作能力的说明，但不关系数据库管理系统语言做具体的语法要求，也就是说不同的关系数据库管理系统可以定义和开发不同的语言来实现关系操作。

2.2.1 基本的关系操作

关系模型中常用的关系操作包括查询(query)操作和更新操作两大部分,而更新操作又可分为插入(insert)、删除(delete)、修改(update)等操作。

关系的查询表达能力很强,因此查询操作是关系操作中最主要的部分。查询操作又可进一步分为选择(select)、投影(project)、连接(join)、除(divide)、并(union)、差(difference)、交(intersection)、笛卡儿积等操作。其中选择、投影、并、差、笛卡儿积是5种基本操作,其他操作可以用基本操作来定义和导出,就像乘法可以用加法来定义和导出一样。

关系操作的特点是集合操作方式,即操作的对象和结果都是集合。这种操作方式也称为成组数据处理(set-at-a-time processing),即一次一个集合的操作方式。相应地,层次模型和网状模型的数据操作方式则为一次一个记录(record-at-a-time)的方式。这里强调一下,关系操作的所有输入和输出均是关系,包括关系操作的中间结果也是关系。

2.2.2 关系数据语言的分类

早期的关系操作能力通常用代数方式或逻辑方式来表示,分别称为关系代数(relational algebra)和关系演算(relational calculus)。关系代数用对关系的运算来表达查询要求,关系演算则用谓词来表达查询要求。关系演算又可按谓词变元的基本对象是元组变量还是域变量分为元组关系演算和域关系演算。一个关系数据语言能够表示关系代数可以表示的查询,称为具有完备的表达能力,简称关系完备性。已经证明关系代数、元组关系演算和域关系演算三种关系数据语言在表达能力上是等价的,都具有完备的表达能力。

关系代数、元组关系演算和域关系演算均是抽象的查询语言,这些抽象的语言与具体的关系数据库管理系统中实现的实际语言并不完全一样。但它们能用作评估实际系统中查询语言能力的标准或基础。实际的查询语言除了提供关系代数或关系演算的功能外,还提供了许多附加功能,例如聚集函数(aggregation function)、关系赋值、算术运算等,使得目前实际数据库系统中查询语言的功能十分强大。

此外,还有一种结构化查询语言(structured query language, SQL)。SQL不仅具有丰富的查询功能,而且具有数据定义和数据控制功能,是集数据查询语言(data query language, DQL)、数据定义语言(DDL)、数据操纵语言(DML)和数据控制语言(data control language, DCL)于一体的关系数据语言。它充分体现了关系数据语言的特点和优点,自20世纪80年代起成为关系数据库的标准语言。

综合来看,关系数据语言可以分为以下几类:

关系数据语言	{	关系代数(如 ISBL)	
		{	元组关系演算语言(如 ALPHA、QUEL)
			域关系演算语言(如 QBE)
	结构化查询语言(SQL),具有关系代数和关系演算双重特点		

特别地, SQL 是一种高度非过程化的语言, 用户不必请求数据库管理员为其建立特殊的存取路径, 存取路径的选择由关系数据库管理系统的优化机制来完成。例如, 在一个存储有千百万条记录的关系中查找符合条件的某一条记录或某一些记录, 从原理上讲可以有多种查找方法。例如, 可以顺序扫描这个关系, 也可以通过某一种索引来查找。不同的查找路径(也称为存取路径)的效率是不同的, 有的完成某一个查询可能很快, 有的可能极慢。关系数据库管理系统研究和开发了查询优化方法, 系统可以自动选择较优的存取路径, 以提高查询效率。

2.3 关系的完整性

关系模型的完整性约束是对关系的某种约束条件。也就是说关系的值随着时间变化时应该满足一些约束条件, 这些约束条件实际上是现实世界的要求。任何关系在任何时刻都要满足这些语义约束。

关系模型中有三类完整性约束: 实体完整性(entity integrity)、参照完整性(referential integrity)和用户定义的完整性(user-defined integrity)。其中实体完整性和参照完整性是关系模型必须满足的完整性约束, 被称作是关系的两个不变性, 应该由关系系统自动支持。用户定义的完整性是应用领域需要遵循的约束条件, 体现了具体领域中的语义约束。

2.3.1 实体完整性

关系数据库中每个元组应该是可区分的, 是唯一的。这样的约束条件用实体完整性来保证。

规则 2.1 实体完整性约束 若属性(指一个或一组属性) A 是基本关系 R 的主属性, 则 A 不能取空值(null value)。所谓空值就是“不知道”或“不存在”或“无意义”的值。有关空值的处理将在第3章 3.5 节空值的处理中详细讲解。

例如, 学生(学号, 姓名, 性别, 出生日期, 主修专业)关系中“学号”为主码并用下划线标识, 则学号取值唯一, 且不能取空值。

按照实体完整性约束的规定, 如果主码由若干属性组成, 则所有这些属性都不能取空值。例如, 学生选课(学号, 课程号, 成绩, 开课学期, 教学班)关系中, “学号, 课程号”为主码, 则“学号”和“课程号”两个属性都不能取空值。

对于实体完整性约束说明如下:

① 实体完整性约束是针对基本关系而言的。一个基本表通常对应现实世界的一个实体集。例如, “学生”关系对应学生的集合。

② 现实世界中的实体是可区分的, 即它们具有某种唯一性标识。例如, 每个学生都是独立的个体, 是不一样的。

③ 相应地, 关系模型中以主码作为唯一性标识。

④ 主码中的属性不能取空值, 如果取了空值, 就说明存在某个不可标识的实体, 即存在

不可区分的实体,这与②相矛盾。因此这个规则称为实体完整性。

2.3.2 参照完整性

现实世界中的实体之间往往存在某种联系,在关系模型中实体及实体间的联系都是用关系来描述的,这样就自然存在着关系与关系间的引用。下面先来看三个示例。

[例 2.1]“学生”实体和“专业”实体可以用下面的关系模式来表示。其中,“专业”实体有 2 个候选码“专业编号”和“专业名”,可以选其中一个为主码,即“专业名”。

学生(学号,姓名,性别,出生日期,主修专业)

专业(专业名,专业编号) /* 专业名、专业编号都是候选码,取专业名为主码 */

这两个关系之间存在着属性的引用,即“学生”关系引用了“专业”关系的主码“专业名”。显然,学生关系中的“主修专业”值必须是确实存在的专业名,即“专业”关系中有该专业的记录。也就是说,“学生”关系中“主修专业”属性的取值需要参照“专业”关系中“专业名”属性的取值。

[例 2.2]“学生”“课程”关系以及两者之间的多对多联系可以用如下三个关系模式表示。

学生(学号,姓名,性别,出生日期,主修专业)

课程(课程号,课程名,学分,先修课)

学生选课(学号,课程号,成绩,开课学期,教学班)

这三个关系模式之间也存在着属性之间的引用,即“学生选课”关系引用了“学生”关系的主码“学号”和“课程”关系的主码“课程号”。同样,“学生选课”关系中的“学号”值必须是确实存在的学生的学号,即“学生”关系中有该学生的记录;“学生选课”关系中的“课程号”值也必须是确实存在的课程的编号,即“课程”关系中有该课程的记录。换句话说,“学生选课”关系中某些属性的取值需要参照其他关系的属性取值。

不仅两个或两个以上的关系间可能存在引用关系,同一关系的内部属性间也可能存在引用关系。

[例 2.3]在课程(课程号,课程名,学分,先修课)关系中,“课程号”属性是主码,“先修课”属性表示选修该门课程之前需要完成先修课程的课程号。它引用了本关系的“课程号”属性,即“课程号”必须是确实存在的课程的编号。

这三个例子说明关系与关系之间、同一关系内部属性之间存在相互引用、相互约束的情况。下面先引入外码的概念,然后给出表达关系之间相互引用约束的参照完整性的定义。

定义 2.5 设 F 是基本关系 R 的一个或一组属性,但不是关系 R 的码, K_s 是基本关系 S 的主码。如果 F 与 K_s 相对应,则称 F 是 R 的外码(foreign key),并称基本关系 R 为参照关系(referencing relation),基本关系 S 为被参照关系(referenced relation)或目标关系(target relation)。关系 R 和 S 不一定是不同的关系。如图 2.2 所示。

显然,目标关系 S 的主码 K_s 和参照关系 R 的外码 F 必须定义在同一个(或同一组)域上。



图 2.2 参照关系与被参照关系示意图

在例 2.1 中，“学生”关系的“主修专业”属性与“专业”关系的主码“专业名”相对应，因此“主修专业”属性是“学生”关系的外码。这里“专业”关系是被参照关系，“学生”关系为参照关系。如图 2.3(a) 所示。

在例 2.2 中，“学生选课”关系的“学号”属性与“学生”关系的主码“学号”相对应，“学生选课”关系的“课程号”属性与“课程”关系的主码“课程号”相对应，因此“学号”和“课程号”是“学生选课”关系的外码。这里“学生”关系和“课程”关系均为被参照关系，“学生选课”关系为参照关系。如图 2.3(b) 所示。

在例 2.3“课程”关系中，“先修课”属性与本身的主码“课程号”属性相对应，因此“先修课”是外码。这里，“课程”关系既是参照关系也是被参照关系。如图 2.3(c) 所示。

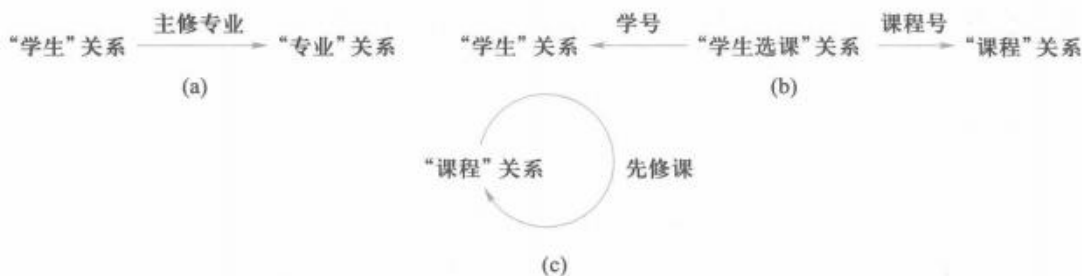


图 2.3 关系的参照图

需要指出的是，外码并不一定要与相应的主码同名，如例 2.1 中“专业”关系的主码为“专业名”，外码为“主修专业”；例 2.3 中“课程”关系的主码为“课程号”，外码为“先修课”。

在实际应用中为了便于识别，当外码与相应的主码属于不同关系时，往往给它们取相同的名字，例如“学生选课”关系中的外码“课程号”与“课程”关系中的主码取相同属性名。

参照完整性约束就是定义外码与主码之间的引用规则。

规则 2.2 参照完整性约束 若属性(或属性组) F 是基本关系 R 的外码，它与基本关系 S 的主码 K_S 相对应(基本关系 R 和 S 不一定是不同的关系)，则对于 R 中每个元组在 F 上的值必须：

- ① 或者取值(F 的每个属性值均为空值)。
- ② 或者等于 S 中某个元组的主码值。

例如，对于例 2.1，“学生”关系中每个元组的“主修专业”属性只能取下面两类值：

- ① 空值，表示该学生尚未选择主修专业。
- ② 非空值，这时该值必须是“专业”关系中某个元组的“专业名”值，表示该学生不可能选

一个不存在的专业。即被参照关系“专业”中一定存在一个元组，它的主码值等于该参照关系“学生”中的外码值。

对于例 2.2，按照参照完整性约束，“学号”和“课程号”属性也可以取两类值：空值或目标关系中已经存在的值。但由于这两个属性是“学生选课”关系中的主属性，按照实体完整性约束，它们均不能取空值，所以“学生选课”关系中的“学号”和“课程号”属性实际上只能取相应被参照关系中已经存在的主码值。

在参照完整性约束中， R 与 S 可以是同一个关系。

例如对于例 2.3，按照参照完整性约束，“先修课”属性值可以取两类值：

- ① 空值，表示该门课程不存在先修课。
- ② 非空值，这时该值必须是本关系中某个元组的课程号。

2.3.3 用户定义的完整性

任何关系数据库系统都应该支持实体完整性和参照完整性，这是关系模型所要求的。除此之外，不同的关系数据库根据其应用场景不同，往往还需要一些特殊的约束条件。用户定义的完整性就是针对某一具体关系数据库的约束条件，它反映某一具体应用所涉及的数据必须满足的语义要求。例如，某个属性必须取唯一值，某个非主属性不能取空值等。例如，在例 2.1 的“学生”关系中，若要求学生必须有姓名，则可以定义“姓名”属性不能取空值；“学生选课”关系中“成绩”的取值范围可以定义为 0~100 等。

关系模型应提供定义和检验这类完整性约束的机制，以使用统一的系统的方法处理它们，而不需要由应用程序承担这一功能。

2.4 关系代数

关系代数是一种抽象的查询语言，它用对关系的运算来表达查询。

任何一种运算都是将一定的运算符作用于一定的运算对象上，得到预期的运算结果。所以运算对象、运算符、运算结果是运算的三大要素。

关系代数的运算对象是关系，运算结果亦为关系。关系代数用到的运算符包括两类：集合运算符和专门的关系运算符，如表 2.4 所示。

按运算符的不同，关系代数的运算可分为传统的集合运算和专门的关系运算两类。其中，传统的集合运算将关系看成元组的集合，其运算是从关系的“水平”方向，即行的角度来进行；专门的关系运算不仅涉及行，而且涉及列。比较运

表 2.4 关系代数运算符

运算符		含义
集合运算符	$\cup$	并
	$-$	差
	$\cap$	交
	$\times$	笛卡儿积
专门的关系运算符	σ	选择
	Π	投影
	$\bowtie$	连接
	$\div$	除

算符和逻辑运算符用于辅助专门的关系运算符进行操作。

2.4.1 传统的集合运算

传统的集合运算是二目运算,包括并、差、交、笛卡儿积4种运算。

设关系 R 和关系 S 具有相同的目 n (即两个关系都有 n 个属性),且相应的属性取自同一个域, t 是元组变量($t \in R$, 表示 t 是 R 的一个元组)。

可以定义并、差、交、笛卡儿积运算如下。

1. 并

关系 R 与关系 S 的并记作

$$R \cup S = \{t | t \in R \vee t \in S\}$$

其结果仍为 n 目关系,由属于 R 或属于 S 的元组组成。

2. 差

关系 R 与关系 S 的差记作

$$R - S = \{t | t \in R \wedge t \notin S\}$$

其结果关系仍为 n 目关系,由属于 R 而不属于 S 的所有元组组成。

3. 交

关系 R 与关系 S 的交记作

$$R \cap S = \{t | t \in R \wedge t \in S\}$$

其结果关系仍为 n 目关系,由既属于 R 又属于 S 的元组组成。关系的交可以用差来表示,即 $R \cap S = R - (R - S)$ 。

4. 笛卡儿积

这里的笛卡儿积严格地讲应该是广义笛卡儿积,因为这里笛卡儿积的元素是元组。

两个分别为 n 目和 m 目的关系 R 和 S ,其笛卡儿积是一个 $n+m$ 列的元组的集合。元组的前 n 列是关系 R 的一个元组,后 m 列是关系 S 的一个元组。若 R 有 k_1 个元组, S 有 k_2 个元组,则关系 R 和关系 S 的笛卡儿积有 $k_1 \times k_2$ 个元组。记作

$$R \times S = \{\widehat{t_i t_j} | t_i \in R \wedge t_j \in S\}$$

图 2.4 为关系 R 、 S 以及两者之间的传统集合运算示例。

2.4.2 专门的关系运算

专门的关系运算包括选择、投影、连接、除等运算。为了叙述上的方便,这里先引入几个记号。

① 设关系模式为 $R(A_1, A_2, \dots, A_n)$, 它的一个关系设为 R 。 $t \in R$, 表示 t 是 R 的一个元组。 $T[A_i]$ 则表示元组 t 中相应于属性 A_i 的一个分量。

② 若 $A = \{A_{i1}, A_{i2}, \dots, A_{ik}\}$, 其中 $A_{i1}, A_{i2}, \dots, A_{ik}$ 是 $A_1, A_2, \dots, A_n$ 的一部分, 则称 A 为属性列或属性组。 $T[A] = (T[A_{i1}], T[A_{i2}], \dots, T[A_{ik}])$ 表示元组 t 在属性列 A 上诸分量的集合, $\bar{A}$ 则

R			S			$R \cup S$		
A	B	C	A	B	C	A	B	C
a_1	b_1	c_1	a_1	b_2	c_2	a_1	b_1	c_1
a_1	b_2	c_2	a_1	b_3	c_2	a_1	b_2	c_2
a_2	b_2	c_1	a_2	b_2	c_1	a_2	b_2	c_1
a_1	b_3	c_2				a_1	b_3	c_2

(a) (b) (c)

$R \cap S$			$R \times S$					
A	B	C	$R.A$	$R.B$	$R.C$	$S.A$	$S.B$	$S.C$
a_1	b_2	c_2	a_1	b_1	c_1	a_1	b_2	c_2
a_2	b_2	c_1	a_1	b_1	c_1	a_1	b_3	c_2
			a_1	b_1	c_1	a_2	b_2	c_1
			a_1	b_2	c_2	a_1	b_2	c_2
			a_1	b_2	c_2	a_1	b_3	c_2
			a_1	b_2	c_2	a_2	b_2	c_1
			a_2	b_2	c_1	a_1	b_2	c_2
			a_2	b_2	c_1	a_1	b_3	c_2
			a_2	b_2	c_1	a_2	b_2	c_1

(d) (e) (f)

$R - S$		
A	B	C
a_1	b_1	c_1

图 2.4 关系 R 、 S 以及两者之间的传统集合运算举例

表示 $\{A_1, A_2, \dots, A_n\}$ 中去掉 $\{A_{i1}, A_{i2}, \dots, A_{ik}\}$ 后剩余的属性列。

③ R 为 n 目关系, S 为 m 目关系。 $t_r \in R$, $t_s \in S$, $\widehat{t_r t_s}$ 称为元组的连接(concatenation)或元组的串接。它是一个 $n+m$ 列的元组, 前 n 个分量为 R 中的一个 n 元组, 后 m 个分量为 S 中的一个 m 元组。

④ 给定一个关系 $R(X, Z)$, X 和 Z 为属性列。当 $t[X] = x$ 时, x 在 R 中的象集(images set)定义为

$$Z_x = \{t[Z] \mid t \in R, t[X] = x\}$$

它表示 R 中属性列 X 上值为 x 的诸元组在 Z 上分量的集合。

例如, 图 2.5 中,

$$x_1 \text{ 在 } R \text{ 中的象集 } Z_{x_1} = \{Z_1, Z_2, Z_3\}$$

$$x_2 \text{ 在 } R \text{ 中的象集 } Z_{x_2} = \{Z_2, Z_3\}$$

$$x_3 \text{ 在 } R \text{ 中的象集 } Z_{x_3} = \{Z_1, Z_3\}$$

R	
x_1	Z_1
x_1	Z_2
x_1	Z_3
x_2	Z_2
x_2	Z_3
x_3	Z_1
x_3	Z_3

图 2.5 象集举例

下面给出这些专门的关系运算的定义。

1. 选择(selection)

选择又称为限制(restriction)。它是在关系 R 中选择满足给定条件的诸元组, 记作

$$\sigma_F(R) = \{t | t \in R \wedge F(t) = \text{'真'}\}$$

其中 F 表示选择条件, 它是一个逻辑表达式, 取逻辑值“真”或“假”。

逻辑表达式 F 的基本形式为

$$X_1 \theta Y_1$$

其中 θ 表示比较运算符, 它可以是 $>$, $\geq$, $<$, $\leq$, $=$ 或 $<>$ 。 X_1 , Y_1 是属性名, 或为常量, 或为简单函数; 属性名也可以用其序号来代替。在基本的选择条件上可以进一步进行逻辑运算, 即进行求非($\neg$)、与($\wedge$)、或($\vee$)运算。条件表达式中的运算符如表 2.5 所示。

表 2.5 条件表达式中的运算符

运算符		含义
比较运算符	$>$	大于
	$\geq$	大于或等于
	$<$	小于
	$\leq$	小于或等于
	$=$	等于
	$<>$	不等于
逻辑运算符	$\neg$	非
	$\wedge$	与
	$\vee$	或

选择运算实际上是从关系 R 中选取使逻辑表达式 F 为真的元组。这是从行的角度进行的运算。

下面对 2.1.3 小节中图 2.1 的“学生选课”数据库中关系 Student、Course 和 SC 进行运算。

[例 2.4] 查询信息安全专业的全体学生。

$$\sigma_{\text{Smajor}=\text{'信息安全'}}(\text{Student})$$

结果如图 2.6(a) 所示。

[例 2.5] 查询 2001 年之后(包含 2001 年)出生的学生。

$$\sigma_{\text{Sbirthdate} \geq 2001-1-1}(\text{Student})$$

结果如图 2.6(b) 所示。

2. 投影(projection)

关系 R 上的投影是从 R 中选择若干属性列组成新的关系。记作

$$\pi_A(R) = \{t[A] | t \in R\}$$

其中 A 为 R 中的属性列。

投影操作是从列的角度进行的运算。

Sno	Sname	Ssex	Sbirthdate	Smajor
20180001	李勇	男	2000-3-8	信息安全

(a) 查询信息安全专业的全体学生

Sno	Sname	Ssex	Sbirthdate	Smajor
20180003	王敏	女	2001-8-1	计算机科学与技术
20180005	陈新奇	男	2001-11-1	信息管理与信息系统
20180007	王佳佳	女	2001-12-7	数据科学与大数据技术

(b) 查询2001年之后(包含2001年)出生的学生

图 2.6 选择运算示例

[例 2.6] 查询学生的学号和主修专业, 即求关系 Student 在“学号”和“主修专业”两个属性上的投影。

$$\Pi_{\text{Sno}, \text{Smajor}}(\text{Student})$$

结果如图 2.7(a) 所示。

投影不仅取消了原关系中的某些属性列, 还可能取消某些元组, 因为取消了某些属性列后就可能出现重复行, 应取消这些完全相同的行。

[例 2.7] 查询关系 Student 中的学生都主修了哪些专业, 即查询关系 Student 在“主修专业”属性上的投影。

$$\Pi_{\text{Smajor}}(\text{Student})$$

结果如图 2.7(b) 所示。关系 Student 原来有 7 个元组, 而投影结果取消了重复的元组, 因此最终结果只有 4 个元组。

学号 Sno	专业 Smajor
20180001	信息安全
20180002	计算机科学与技术
20180003	计算机科学与技术
20180004	计算机科学与技术
20180005	信息管理与信息系统
20180006	数据科学与大数据技术
20180007	数据科学与大数据技术

(a) 查询学生的学号和主修专业

专业 Smajor
信息安全
计算机科学与技术
信息管理与信息系统
数据科学与大数据技术

(b) 查询学生主修的专业

图 2.7 投影运算示例

3. 连接(join)

连接也称为 θ 连接, 指从两个关系的笛卡儿积中选取其属性间满足一定条件的元组。记作

$$R \bowtie_{A\theta B} S = \{ \widehat{t_r t_s} \mid t_r \in R \wedge t_s \in S \wedge t_r[A] \theta t_s[B] \}$$

其中, A 和 B 分别为关系 R 和 S 上列数相等且可比的属性列, θ 是比较运算符。具体来说, 连接运算是从笛卡儿积 $R \times S$ 中选取关系 R 在属性列 A 上的值与关系 S 在属性列 B 上的值满足比较关系 θ 的元组。

连接运算中有两种最为重要也最为常用的连接, 一种是等值连接(equijoin), 另一种是自然连接(natural join)。

θ 为“=”的连接运算称为等值连接。它是从关系 R 与 S 的广义笛卡儿积中选取 A 、 B 属性值相等的那些元组, 即

$$R \bowtie_{A=B} S = \{ \widehat{t_r t_s} \mid t_r \in R \wedge t_s \in S \wedge t_r[A] = t_s[B] \}$$

自然连接是一种特殊的等值连接。它要求两个关系中进行比较的分量必须是同名的属性列, 并且在结果中把重复的属性列去掉。即若 R 和 S 中具有相同的属性列 B , U 为 R 和 S 的全体属性集合, 则自然连接可记作

$$R \bowtie S = \{ \widehat{t_r t_s} [U-B] \mid t_r \in R \wedge t_s \in S \wedge t_r[B] = t_s[B] \}$$

一般的连接操作是从行的角度进行运算, 但自然连接还需要取消重复属性列, 所以是同时从行和列的角度进行运算。

[例 2.8] 对于图 2.8(a)(b)所示的关系 R 和 S , 图 2.8(c)(d)(e)分别给出了非等值连接 $R \bowtie_{C < E} S$ 、等值连接 $R \bowtie_{R.B=S.B} S$ 和自然连接 $R \bowtie S$ 的结果。

R			S		$R \bowtie_{C < E} S$				
A	B	C	B	E	A	$R.B$	C	$S.B$	E
a_1	b_1	5	b_1	3	a_1	b_1	5	b_2	7
a_1	b_2	6	b_2	7	a_1	b_1	5	b_3	10
a_2	b_3	8	b_3	10	a_1	b_2	6	b_2	7
a_2	b_4	12	b_3	2	a_1	b_2	6	b_3	10
			b_5	2	a_2	b_3	8	b_3	10

(a) 关系 R

(b) 关系 S

(c) 非等值连接

$R \bowtie_{R.B=S.B} S$					$R \bowtie S$			
A	$R.B$	C	$S.B$	E	A	B	C	E
a_1	b_1	5	b_1	3	a_1	b_1	5	3
a_1	b_2	6	b_2	7	a_1	b_2	6	7
a_2	b_3	8	b_3	10	a_2	b_3	8	10
a_2	b_3	8	b_3	2	a_2	b_3	8	2

(d) 等值连接

(e) 自然连接

图 2.8 连接运算举例

两个关系 R 和 S 在做自然连接时, 选择两个关系在公共属性上值相等的元组构成新的关系。此时, 关系 R 中某些元组有可能在 S 中不存在公共属性上值相等的元组, 从而造成 R 中这些元组在操作时被舍弃了; 同样, S 中某些元组也可能被舍弃。这些被舍弃的元组称为悬浮元组(dangling tuple)。例如, 在图 2.8(e)的自然连接中, R 中的第 4 个元组、 S 中的第 5 个元组都是被舍弃的悬浮元组。

如果把悬浮元组也保存在结果关系中, 而在其他属性上填空值(NULL), 那么这种连接就叫作外连接(outer join), 记作 $R \bowtie S$; 如果只保留左边关系 R 中的悬浮元组就叫作左外连接(left outer join 或 left join), 记作 $R \ltimes S$; 如果只保留右边关系 S 中的悬浮元组就叫作右外连接(right outer join 或 right join), 记作 $R \rtimes S$ 。

例如, 图 2.9(a)是图 2.8 中关系 R 和关系 S 的外连接, 图 2.9(b)是左外连接, 图 2.9(c)是右外连接。

A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
a_2	b_4	12	NULL
NULL	b_5	NULL	2

(a) 外连接

A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
a_2	b_4	12	NULL

(b) 左外连接

A	B	C	E
a_1	b_1	5	3
a_1	b_2	6	7
a_2	b_3	8	10
a_2	b_3	8	2
NULL	b_5	NULL	2

(c) 右外连接

图 2.9 外连接运算举例

4. 除 (division)

设关系 R 除以关系 S 的结果为关系 T , 则 T 包含所有在 R 但不在 S 中的属性及其值, 且 T 的元组与 S 的元组的所有组合都在 R 中。

下面用象集来定义除法。

给定关系 $R(X, Y)$ 和 $S(Y, Z)$, 其中 X, Y, Z 为属性列。 R 中的 Y 与 S 中的 Y 可以有不同的属性名, 但必须出自相同的域。

R 与 S 的除运算得到一个新的关系 $P(X)$, P 是 R 中满足下列条件的元组在 X 属性列上的投影: 元组在 X 上分量值 x 的象集 Y_x 包含 S 在 Y 上投影的集合。记作

$$R \div S = \{t_r[X] \mid t_r \in R \wedge \Pi_Y(S) \subseteq Y_x\}$$

其中 Y_x 为 x 在 R 中的象集, $x = t_r[X]$ 。

除操作是同时从行和列角度进行运算。

[例 2.9] 设关系 R, S 分别为图 2.10 中的(a)和(b), $R \div S$ 的结果为图 2.10(c)。

在关系 R 中, A 可以取 4 个值 $\{a_1, a_2, a_3, a_4\}$ 。其中:

$$a_1 \text{ 的象集为 } \{(b_1, c_2), (b_2, c_3), (b_2, c_1)\}$$

a_2 的象集为 $\{(b_3, c_7), (b_2, c_3)\}$

a_3 的象集为 $\{(b_4, c_6)\}$

a_4 的象集为 $\{(b_6, c_6)\}$

S 在 (B, C) 上的投影为 $\{(b_1, c_2), (b_2, c_1), (b_2, c_3)\}$ 。

显然只有 a_1 的象集 $(B, C)_{a_1}$ 包含了 S 在 (B, C) 属性列上的投影, 所以

$$R \div S = \{a_1\}$$

R			S		
A	B	C	B	C	D
a_1	b_1	c_2	b_1	c_2	d_1
a_2	b_3	c_7	b_2	c_1	d_1
a_3	b_4	c_6	b_2	c_3	d_2
a_1	b_2	c_3			
a_4	b_6	c_6			
a_2	b_2	c_3			
a_1	b_2	c_1			

(a)

R÷S		
A		
a_1		

(c)

图 2.10 除运算举例

图 2.11 为例 2.9 的除运算过程示意。

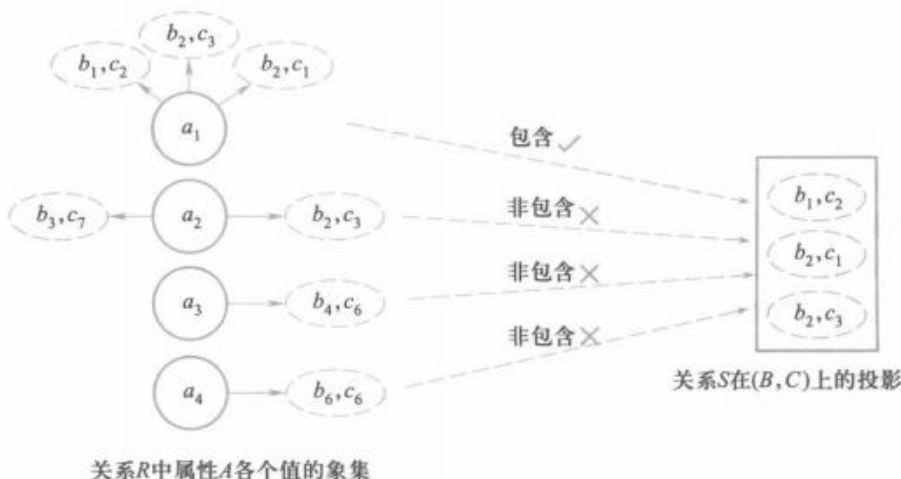


图 2.11 例 2.9 的除运算过程示意图

下面仍以 2.1.3 小节图 2.1 “学生选课”数据库为例, 给出几个综合应用多种关系代数运算进行查询的例子。

[例 2.10] 查询至少选修 81001 号课程和 81003 号课程的学生的学号。

首先建立一个临时关系 K :

K	
Cno	
81001	
81003	

然后求:

$$\Pi_{Sno, Cno}(SC) \div K = \{20180001, 20180002\}$$

求解过程与例 2.9 类似, 先对关系 SC 在 (Sno, Cno) 属性上投影, 然后逐一求出每一学生 (Sno) 的象集, 并依次检查这些象集是否包含 K 。

[例 2.11] 查询选修了 81002 号课程的学生的学号。

$$\Pi_{Sno}(\sigma_{Cno='81002'}(SC)) = \{20180001, 20180002, 20180003\}$$

[例 2.12] 查询至少选修了一门其直接先修课为 81003 号课程的学生的姓名。

$$\Pi_{Sname}(\sigma_{Cpno='81003'}(Course) \bowtie SC \bowtie \Pi_{Sno, Sname}(Student))$$

或

$$\Pi_{Sname}(\Pi_{Sno}(\sigma_{Cpno='81003'}(Course) \bowtie SC) \bowtie \Pi_{Sno, Sname}(Student))$$

[例 2.13] 查询选修了全部课程的学生的学号和姓名。

$$\Pi_{Sno, Cno}(SC) \div \Pi_{Cno}(Course) \bowtie \Pi_{Sno, Sname}(Student)$$

关系代数小结

本节介绍了 8 种关系代数运算, 其中并、差、笛卡儿积、选择和投影这 5 种运算为基本的运算。其他三种运算, 即交、连接和除, 均可以用这 5 种基本运算来表达。引进它们并不增加语言的运算能力, 但可以简化表达。

关系代数中, 这些运算经有限次复合而形成的表达式称为关系代数表达式。

此外, 还有扩展的关系代数, 如关系的重新命名、查询结果的去重操作、分组操作、排序操作和聚集函数等, 这里就不介绍了。

* 2.5 关系演算

关系演算是以数理逻辑中的谓词演算为基础的。按谓词变元不同, 关系演算可分为元组关系演算和域关系演算。本节通过元组关系演算语言 ALPHA 和域关系演算语言 QBE 来介绍关系演算的基本概念和查询方法。

* 2.5.1 元组关系演算语言 ALPHA

元组关系演算以元组变量作为谓词变元的基本对象。一种典型的元组关系演算语言是 Edgar F. Codd 提出的 ALPHA。这一语言虽然没有实际实现, 但关系数据库管理系统 INGRES 最初所用的 QUEL 语言就是参照 ALPHA 研制的, 与其十分类似。

ALPHA 语言主要有 GET、PUT、HOLD、UPDATE、DELETE、DROP 这 6 条语句。其语句的基本格式如下。

操作语句 工作空间名(表达式): 操作条件

其中,“表达式”用于指定语句的操作对象,它可以是关系名或(和)属性名,一条语句可以同时操作多个关系或多个属性。“操作条件”是一个逻辑表达式,用于将操作结果限定在满足条件的元组中,操作条件可以为空。除此之外,还可以在基本格式的基础上加上排序要求以及指定返回元组的条数等。

1. 检索操作

检索操作用 GET 语句实现。

(1) 简单检索(即不带条件的检索)

[例 2.14] 查询所有被选修的课程的课程号。

```
GET W(SC. Cno)
```

其中,“W”为工作空间名。这里条件为空,表示没有限定条件。

[例 2.15] 查询所有学生的数据。

```
GET W(Student)
```

(2) 限定的检索(即带条件的检索)

[例 2.16] 查询计算机科学与技术专业 2000 年之前出生的学生的学号和出生日期。

```
GET W(Student. Sno, Student. Sbirthdate) : Student. Smajor='计算机科学与技术'
      ^ Student. Sbirthdate<'2000-1-1'
```

(3) 带排序的检索

[例 2.17] 查询计算机科学与技术专业学生的学号和出生日期,结果按出生日期降序排序。

```
GET W(Student. Sno, Student. Sbirthdate) : StudentSmajo='计算机科学与技术'
      DOWN Student. Sbirthdate
```

其中,“DOWN”表示降序排序。

(4) 指定返回元组的条数的检索

[例 2.18] 取出一个信息安全专业学生的学号。

```
GET W(1)(Student. Sno) : Student. Smajor='信息安全'
```

其中,“W”后括号中的数量就是指定的返回元组的个数。

[例 2.19] 查询选修了 81003 号课程、成绩在前三名的学生的学号及其成绩。

```
GET W(3)(SC. Sno, SC. Grade) : SC. Cno='81003' DOWN SC. Grade
```

(5) 用元组变量的检索

前面已讲到,元组关系演算是以元组变量作为谓词变元的基本对象。元组变量是在某一关

系范围内变化的, 所以也称为范围变量(range variable), 一个关系可以设多个元组变量。

元组变量主要有两方面的用途:

① 简化关系名。如果关系名很长, 使用起来感到不方便, 则可以设一个较短名字的元组变量来代替关系名。

② 操作条件中使用量词时必须用元组变量。

[例 2.20] 查询信息安全专业学生的姓名。

```
RANGE Student X
```

```
GET W(X. Sname) : X. Smajor='信息安全'
```

ALPHA 语言用 RANGE 来说明元组变量。本例中 X 是关系 Student 上的元组变量, 用途是简化关系名, 即用 X 代表 Student。

(6) 用存在量词(existential quantifier)的检索

例 2.21、例 2.22、例 2.23 中的元组变量都是为存在量词而设的。其中例 2.23 需要对两个关系使用存在量词, 所以设了两个元组变量。

[例 2.21] 查询选修了 81002 号课程的学生的姓名。

```
RANGE SC X
```

```
GET W(Student. Sname) :  $\exists X(X. Sno=Student. Sno \wedge X. Cno='81002')$ 
```

[例 2.22] 查询选修了直接先修课程是 81002 号课程的学生的学号。

```
RANGE Course CX
```

```
GET W(SC. Sno) :  $\exists CX(CX. Cno=SC. Cno \wedge CX. Cjno='81002')$ 
```

[例 2.23] 查询至少选修一门其先修课为 81002 号课程的学生的姓名。

```
RANGE Course CX
```

```
SC SCX
```

```
GET W(Student. Sname) :  $\exists SCX(SCX. Sno=Student. Sno \wedge$   

 $\exists CX(CX. Cno=SCX. Cno \wedge CX. Cjno='81002'))$ 
```

在本例中的元组关系演算公式可以变换为前束范式(prenex normal form)的形式:

```
RANGE Course CX
```

```
SC SCX
```

```
GET W(Student. Sname) :  $\exists SCX \exists CX(SCX. Sno=Student. Sno \wedge$   

 $CX. Cno=SCX. Cno \wedge CX. Cjno='81002')$ 
```

(7) 带有多个关系的表达式的检索

上面所举的各个例子中, 虽然查询时可能会涉及多个关系, 即公式中可能涉及多个关系, 但查询结果表达式中只有一个关系。实际上表达式中是可以有多个关系的。

[例 2.24] 查询成绩为 90 分以上的学生姓名与课程名称。

本查询所要求的结果是学生姓名和课程名称, 分别在 Student 和 Course 两个关系中。

RANGE SC SCX

GET W(Student, Sname, Course, Cname) : $\exists$ SCX (SCX. Grade $\geq 90 \wedge$
SCX. Sno = Student. Sno $\wedge$ Course. Cno = SCX. Cno)

(8) 用全称量词(universal quantifier)的检索

[例 2.25] 查询不选 81004 号课程的学生姓名。

RANGE SC SCX

GET W(Student, Sname) : $\forall$ SCX (SCX. Sno $\neq$ Student. Sno $\vee$ SCX. Cno $\neq$ '81004')

本例也可以用存在量词来表示:

RANGE SC SCX

GET W(Student, Sname) : $\neg \exists$ SCX (SCX. Sno = Student. Sno $\wedge$ SCX. Cno = '81004')

(9) 用两种量词的检索

[例 2.26] 查询选修了全部课程的学生姓名。

RANGE Course CX

SC SCX

GET W(Student, Sname) : $\forall$ CX $\exists$ SCX (SCX. Sno = Student. Sno $\wedge$ SCX. Cno = CX. Cno)

(10) 用蕴涵(implication)的检索

[例 2.27] 查询至少选修了 20180003 号学生所选课程的学生学号。

本例题的求解思路是, 对 Course 中的所有课程依次检查每一门课程, 看 20180003 是否选修了该课程, 如果选修了, 则再看某一个学生是否也选修了该门课。如果对于 20180003 所选的每门课程该学生都选修了, 则该学生为满足要求的学生。把所有这样的学生全都找出来即完成了本题。

RANGE Course CX

SC SCX /* 注意, 这里 SC 设了两个元组变量 */

SC SCY

GET W(Student, Sno) : $\forall$ CX ($\exists$ SCX (SCX. Sno = '20180003' $\wedge$ SCX. Cno = CX. Cno)
 $\Rightarrow \exists$ SCY (SCY. Sno = Student. Sno $\wedge$ SCY. Cno = CX. Cno))

(11) 聚集函数

用户在使用查询语言时经常要做一些简单的计算, 例如要计算符合某一查询要求的元组数, 计算某个关系中所有元组在某属性上的值的总和或平均值等。为了方便用户, 关系数据语言中建立了有关这类运算的标准函数库供用户选用。这类函数通常称为聚集函数或内置函数(built-in function)。关系演算中提供了 COUNT、TOTAL、MAX、MIN、AVG 等聚集函数, 其含义如表 2.6 所示。

表 2.6 关系演算中的聚集函数

函数名	功能
COUNT	对元组计数
TOTAL	求总和
MAX	求最大值
MIN	求最小值
AVG	求平均值

[例 2.28] 查询学生所选的专业的数目。

```
GET W( COUNT( Student. Smajor ) )
```

COUNT 函数在计数时会自动排除重复值。

[例 2.29] 查询 2020 年第二学期选修了 81003 号课程的学生的平均成绩。

```
GET W( AVG( SC. Grade ) ) : SC. Cno = '81003' ^ SC. Semester = '20202'
```

2. 更新操作

(1) 修改操作

修改操作用 UPDATE 语句实现。其步骤是：

- ① 用 HOLD 语句将要修改的元组从数据库读到工作空间中。
- ② 用宿主语言修改工作空间中元组的属性值。
- ③ 用 UPDATE 语句将修改后的元组送回数据库。

需要注意的是，单纯检索数据使用 GET 语句即可，但为修改数据而读元组时必须使用 HOLD 语句，HOLD 语句是带并发控制的 GET 语句。有关并发控制的概念将在第 12 章并发控制中详细介绍。

[例 2.30] 把学号为 20180004 的学生从计算机科学与技术专业转到信息管理与信息系统专业。

```
HOLD W( Student. Sno, Student Smajor ) : Student. Sno = '20180004'
/* 从 Student 关系中读出学号为 20180004 的学生的数据 */
MOVE '信息管理与信息系统' TO W. Smajor /* 用宿主语言进行修改 */
UPDATE W /* 把修改后的元组送回 Student 关系 */
```

在该例中用 HOLD 语句来读 20180004 号学生的数据，而不是用 GET 语句。

如果修改操作涉及两个关系，就要执行两次 HOLD-MOVE-UPDATE 操作序列。

在 ALPHA 语言中，修改关系主码的操作是不允许的，例如不能用 UPDATE 语句将学号“20180004”改为“20180009”。如果需要修改主码值，只能先用删除操作删除该元组，然后再把具有新主码值的元组插入关系。

(2) 插入操作

插入操作用 PUT 语句实现。其步骤是：

- ① 用宿主语言在工作空间中建立新元组。
- ② 用 PUT 语句将该元组插入指定的关系。

[例 2.31] 学校新开设了一门 2 学分的课程“计算机组织与结构”，其课程号为 81009，直接先行课为 81005 号课程。插入该课程元组。

```
MOVE '81009' TO W. Cno
MOVE '计算机组织与结构' TO W. Cname
```

```
MOVE '2' TO W. Ccredit  
MOVE '81005' TO W. Cpno  
PUT W( Course)      /* 把 W 中的元组插入指定关系 Course */
```

PUT 语句只对一个关系操作,也就是说表达式必须为单个关系名。

(3) 删除

删除操作用 DELETE 语句实现。其步骤为:

- ① 用 HOLD 语句把要删除的元组从数据库读到工作空间中。
- ② 用 DELETE 语句删除该元组。

[例 2.32] 20180007 号学生因故退学,删除该学生元组。

```
HOLD W( Student) : Student. Sno = '20180007'  
DELETE W
```

[例 2.33] 将学号 20180001 改为 20180010。

```
HOLD W( Student) : Student. Sno = '20180001'  
DELETE W  
MOVE '20180010' TO W. Sno  
MOVE '李勇' TO W. Sname  
MOVE '男' TO W. Ssex  
MOVE '2000-3-8' TO W. Sbirthdate  
MOVE '信息安全' TO W. Smajor  
PUT W( Student)
```

[例 2.34] 删除全部学生。

```
HOLD W( Student)  
DELETE W
```

由于 SC 关系与 Student 关系之间具有参照关系,为保证参照完整性,删除 Student 中的元组时要相应地删除 SC 中的元组(手工删除或由数据库管理系统自动删除)。

```
HOLD W( SC)  
DELETE W
```

ALPHA 语言例题解析可参见二维码内容。



扩展阅读: ALPHA
语言例题解析

* 2.5.2 域关系演算语言 QBE

关系演算的另一种形式是域关系演算。域关系演算以元组变量的分量(即域变量)作为谓词变元的基本对象。1975 年由 M. M. Zloof 提出的 QBE 就是一个很有特色的域关系演算语言,

该语言于 1978 年在 IBM 370 上得以实现。

QBE 是 Query By Example(即通过例子进行查询)的简称,它最突出的特点是操作方式。它是一种高度非过程化的基于屏幕表格的查询语言,用户通过终端屏幕编辑程序,以填写表格的方式构造查询要求,而查询结果也是以表格形式显示,因此非常直观、易学易用。

QBE 用示例元素来表示查询结果可能的情况,示例元素实质上就是域变量。QBE 的操作框架如图 2.12 所示。

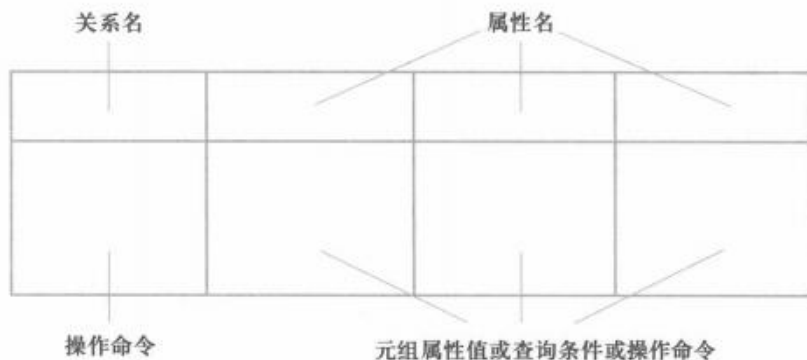


图 2.12 QBE 的操作框架

下面仍以 2.1.3 小节图 2.1 所示的“学生选课”关系数据库为例,说明 QBE 的用法。

1. 检索操作

(1) 简单查询

[例 2.35]求信息安全专业全体学生的姓名。

操作步骤如下:

- ① 用户提出要求。
- ② 屏幕显示如下空白表格。

- ③ 用户在最左边一栏输入关系名 Student。

Student					

- ④ 系统显示该关系的属性名。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor

⑤ 用户在上面构造查询要求。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
		P. <u>T</u>			信息安全

这里“T”是示例元素，即域变量。QBE 要求示例元素下面一定要加下划线。“信息安全”是查询条件，不用加下划线。“P.”是操作符，表示打印(print)，实际上是显示。

查询条件中可以使用比较运算符>、≥、<、≤、=和≠，其中=可以省略。

示例元素是这个域中可能的一个值，它不必是查询结果中的元素。比如若求信息系的学生，只要给出任意的一个学生名即可，而不必真是信息系的某个学生名。

对于例 2.35，可构造查询要求如下：

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
		P. <u>刘晨</u>			信息安全

这里的查询条件是 Smajor='信息安全'，其中“=”被省略。

⑥ 屏幕显示查询结果。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
		<u>李勇</u>			信息安全

即根据用户的查询要求找出信息系的学生姓名。

[例 2.36] 查询全体学生的全部数据。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	P. <u>20180002</u>	P. <u>刘晨</u>	P. <u>女</u>	P. <u>1999-9-1</u>	P. <u>计算机科学与技术</u>

显示全部数据也可以简单地把 P. 操作符作用在关系名上。因此本查询也可以简单地表示如下：

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
P.					

(2) 条件查询

[例 2.37] 求出生日期晚于 1999-9-1 的学生的学号。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	P. <u>20180002</u>			>1999-9-1	

[例 2.38] 求计算机科学与技术专业出生日期晚于 1999-9-1 的学生的学号。

本查询的条件是 Smajor='计算机科学与技术'和 Sbirthdate>'1999-9-1'两个条件的“与”。在 QBE 中，表示两个条件的“与”有两种方法：

① 把两个条件写在同一行上。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	P. 20180002			>1999-9-1	计算机科学与技术

② 把两个条件写在不同行上，但使用相同的示例元素值。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	P. 20180002				计算机科学与技术
	P. 20180002			>1999-9-1	

[例 2.39] 查询计算机科学与技术专业或者出生日期晚于 1999-9-1 的学生的学号。

本查询的条件是 Smajor='计算机科学与技术'和 Sbirthdate>'1999-9-1'两个条件的“或”。在 QBE 中把两个条件写在不同行上，并且使用不同的示例元素值，即表示条件的“或”。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	P. 20180002				计算机科学与技术
	P. 20180003			>1999-9-1	

对于多行条件的查询，先输入哪一行是任意的，不影响查询结果。这就允许用户以不同的思考方式进行查询，十分灵活、自由。

[例 2.40] 查询既选修了 81001 号课程又选修了 81002 号课程的学生的学号。

本查询条件是在一个属性中的“与”关系，它只能用“与”条件的方法②表示，即写两行，但示例元素相同。

SC	Sno	Cno	Grade	Semester	Teachingclass
	P. 20180002	81001			
	P. 20180002	81002			

[例 2.41] 查询选修 81001 号课程的学生的姓名。

本查询涉及 SC 和 Student 两个关系。在 QBE 中实现这种查询的方法是通过相同的连接属性值把多个关系连接起来。这里示例元素 Sno 是连接属性，其值在两个表中要相同。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	20180002	P. 李勇			

SC	Sno	Cno	Grade	Semester	Teachingclass
	20180002	81001			

[例 2.42] 查询未选修 81001 号课程的学生的姓名。

这里的查询条件中用到逻辑“非”。在 QBE 中表示逻辑“非”的方法是将运算符“¬”写在关系名下面。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	<u>2018002</u>	P. 李勇			
	<u>2018002</u>	P. 李勇			

SC	Sno	Cno	Grade	Semester	Teachingclass
┐	<u>2018002</u>	81001			
┐	<u>2018002</u>				

这个查询就是显示学号示例为“2018002”的学生的姓名，而该学生选修 81001 号课程的情况为假或者什么课程都没有选修。

[例 2.43] 查询有两个人以上选修的课程课程号。

本查询是在一个表内连接。这个查询就是要显示这样的课程号“81001”，它不仅被学生“2018002”选修，而且也被另一个学生“┐ 2018002”选修了。

SC	Sno	Cno	Grade	Semester	Teachingclass
	<u>2018002</u>	P. <u>81001</u>			
	┐ <u>2018002</u>	<u>81001</u>			

(3) 聚集函数

为了方便用户，QBE 提供了一些聚集函数，主要包括 CNT、SUM、AVG、MAX、MIN 等，其含义如表 2.7 所示。

表 2.7 QBE 中的聚集函数

函数名	功能
CNT	对元组计数
SUM	求总和
AVG	求平均值
MAX	求最大值
MIN	求最小值

[例 2.44] 查询信息安全专业学生的总人数。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	P. <u>CNT</u>			.	信息安全

(4) 对查询结果排序

对查询结果按某个属性值的升序排序，只需在相应列中填入“AO.”，按降序排序则填“DO.”。如果按多列排序，用“AO(*i*).”或“DO(*i*).”表示，其中 *i* 为排序的优先级，*i* 值越小，优先级越高。

[例 2.45] 查询全体男生的姓名，要求查询结果按专业名升序排序，对同一专业的学生按出生日期降序排序。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
		P. 李勇	男	DO(2).	AO(1).

2. 更新操作

(1) 修改操作

修改操作符为“U.”。QBE 不允许修改关系的主码，如果需要修改某个元组的主码，只能先删除该元组，然后再插入新主码的元组。

[例 2.46] 把学号为 20180001 的学生的出生日期改为“2001-3-8”。

这是一个简单修改操作，不包含算术表达式，因此可以有两种表示方法：

① 将操作符“U.”放在值上。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	20180001			U. 2001-3-8	

② 将操作符“U.”放在关系上。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
U.	20180001			2001-3-8	

这里，码“20180001”标明要修改的元组。“U.”标明是修改后的新值。由于主码是不能修改的，所以即使在第二种写法中，系统也不会混淆要修改的属性值。

[例 2.47] 将学号为 20180004 的学生在 2019 年第 2 学期选修 81001 号课程的成绩增加 6 分。

这个修改操作涉及表达式，所以只能将操作符“U.”放在关系上。

SC	Sno	Cno	Grade	Semester	Teachingclass
	20180004	81001	56	20192	
U.	20180004		56+6		

[例 2.48] 将 2019 年第 2 学期选修 81001 号课程的学生们的成绩都增加 5 分。

SC	Sno	Cno	Grade	Semester	Teachingclass
	<u>20180001</u>	81001	<u>70</u>	20192	
U.	<u>20180001</u>		<u>70+5</u>	20192	

(2) 插入操作

插入操作符为“I.”。新插入的元组必须具有主码值，其他属性值可以为空。

[例 2.49] 把计算机科学与技术专业的学生“高大卫，男，学号 20190006，出生日期 2001-6-28”存入数据库。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
I.	20190006	高大卫	男	2001-6-28	计算机科学与技术

(3) 删除操作

删除操作符为“D.”。

[例 2.50] 删除学号为 20180006 的学生。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
D.	20180006				

由于 SC 关系与 Student 关系之间具有参照关系, 为保证参照完整性, 删除学号为 20180006 的学生后, 通常还应删除该学生选修的全部课程。

SC	Sno	Cno	Grade	Semester	Teachingclass
D.	20180006				

本章小结

关系模型是关系数据库的核心概念和基础, 关系数据库系统是目前使用最广泛的数据库系统。20 世纪 70 年代以后开发的数据库管理系统产品几乎都是基于关系模型的。在数据库发展的历史上, 最重要的创新成果之一就是关系模型。

关系模型与层次模型、网状模型最重要的区别是, 关系模型只有“表”这一种数据结构, 而层次模型、网状模型还有其他数据结构, 以及对这些数据结构的操作。关系模型的数据结构简单清晰, 用户容易理解, 又具有坚实的数学基础。



扩展阅读: 元组
关系演算表达式

本章系统地讲解了关系模型的重要概念, 包括关系模型的数据结构、关系操作以及关系的三类完整性约束; 介绍了用代数方式和逻辑方式来表达的关系语言, 即关系代数、元组关系演算和域关系演算。在关系演算中介绍了元组关系演算语言 ALPHA 和域关系演算语言 QBE。有关元组关系演算表达式的内容, 感兴趣的读者可参见二维码介绍。

习 题 2

1. 试述关系模型的三个组成部分。
2. 简述关系数据语言的特点和分类。
3. 定义并理解下列术语, 说明它们之间的联系与区别:
 - ① 域, 笛卡儿积, 关系, 元组, 属性;
 - ② 主码, 全码, 候选码, 外码, 主属性、非主属性;
 - ③ 关系模式, 关系, 关系数据库。
4. 举例说明关系模式和关系的区别。
5. 试述关系模型的完整性约束。在参照完整性中, 什么情况下外码属性的值可以为空值?
6. 设有一个 SPJ 数据库, 包括 4 个关系模式 S、P、J 和 SPJ。

S(SNO, SNAME, STATUS, CITY);

P(PNO, PNAME, COLOR, WEIGHT);

J(JNO,JNAME,CITY);
SPJ(SNO,PNO,JNO,QTY)。

供应商表 S 由供应商代码(SNO)、供应商姓名(SNAME)、供应商状态(STATUS)、供应商所在城市(CITY)组成。

零件表 P 由零件代码(PNO)、零件名(PNAME)、颜色(COLOR)、重量(WEIGHT)组成。

工程项目表 J 由工程项目代码(JNO)、工程项目名(JNAME)、工程项目所在城市(CITY)组成。

供应情况表 SPJ 由供应商代码(SNO)、零件代码(PNO)、工程项目代码(JNO)、供应数量(QTY)组成,表示某供应商供应某种零件给某工程项目的数量为 QTY。

今有若干数据如图 2.13 所示。

S

SNO	SNAME	STATUS	CITY
S1	精益	20	天津
S2	盛锡	10	北京
S3	东方红	30	北京
S4	丰泰盛	20	天津
S5	为民	30	上海

P

PNO	PNAME	COLOR	WEIGHT
P1	螺母	红	12
P2	螺栓	绿	17
P3	螺丝刀	蓝	14
P4	螺丝刀	红	14
P5	凸轮	蓝	40
P6	齿轮	红	30

J

JNO	JNAME	CITY
J1	三建	北京
J2	一汽	长春
J3	弹簧厂	天津
J4	造船厂	天津
J5	机车厂	唐山
J6	无线电厂	常州
J7	半导体厂	南京

SPJ

SNO	PNO	JNO	QTY
S1	P1	J1	200
S1	P1	J3	100
S1	P1	J4	700
S1	P2	J2	100
S2	P3	J1	400
S2	P3	J2	200
S2	P3	J4	500
S2	P3	J5	400
S2	P5	J1	400
S2	P5	J2	100
S3	P1	J1	200
S3	P3	J1	200
S4	P5	J1	100
S4	P6	J3	300
S4	P6	J4	200
S5	P2	J4	100
S5	P3	J1	200
S5	P6	J2	200
S5	P6	J4	500

图 2.13 SPJ 数据库数据

试用关系代数、元组关系演算语言 ALPHA 和域关系演算语言 QBE 完成如下查询:

- ① 求供应工程 J1 零件的供应商代码 SNO。
- ② 求供应工程 J1 零件 P1 的供应商代码 SNO。
- ③ 求供应工程 J1 零件为红色的供应商代码 SNO。
- ④ 求没有使用天津供应商生产的红色零件的工程号 JNO。

- ⑤ 求至少使用了与供应商 S1 所供应的全部零件相同零件号的工程号 JNO。
7. 试述等值连接与自然连接的区别和联系。
8. 关系代数的基本运算有哪些? 如何用这些基本运算来表示其他运算?

参考文献 2

[1] CODD E F. A relational model of data for large shared data banks[J]. CACM-Communications of the ACM, 1970, 13(6): 377-387.

1970 年, Edgar F. Codd 在文献[1]中首先提出了关系数据模型, 以后 Codd 又提出了关系代数和关系演算的概念, 以及函数依赖的概念, 1972 年提出了关系的第一、第二、第三范式, 1974 年提出了 BC(Boyce-Codd) 范式, 为关系数据库系统奠定了理论基础。由于对关系数据库理论的突出贡献, 1981 年 Codd 获得了图灵奖。

[2] CODD E F. A data base sublanguage founded on the relational calculus[C]. Proceedings of the ACM SIGMOD Workshop on Data Description, Access and Control, 1971: 35-68.

[3] CODD E F. Relational completeness of database sublanguages in data base systems[R]. Courant Computer Science Symposia Series, 1972(6).

[4] CODD E F. Further normalization of the data base relational model[M]. Data Base Systems. Prentice-Hall, 1972.

[5] ULLMAN J D. Principles of database systems[M]. 2nd ed. Computer Science Press, 1982.

Ullman 在文献[5]中给出了关系代数、元组关系演算和域关系演算的等价性证明。

[6] ZLOOF M M. Query By Example[C]. Proceeding of the National Computer Conference and Exposition(AFIPS'75), 1975: 431-438.

[7] LACROIX M, PIROTTE A. Domain-oriented relational language. Proceedings of the 3rd International Conference on Very Large Data Bases[C], 1977: 370-378.

[8] LACROIX M, PIROTTE A. ILL: an English structured query language for relational data bases[J]. ACM SIGART Bulletin, 1977, 61: 61-63.

域关系演算的思想出现在文献[7]描述的 QBE 语言中, 形式化定义由 Lacroix 和 Pirotte 在文献[8]中给出。文献[8]给出了一个域关系演算语言 ILL。

[9] STONEBRAKER M, WONG E, KREPS P, et al. The design and implementation of INGRES[J]. ACM Transactions on Database Systems, 1976, 1(3): 189-222.

[10] DATE C J. Referential integrity[C]. Proceedings of the 7th International Conference on Very Large Data Bases, 1981, 7: 2-12.

[11] DATE C J. Why relations should have exactly one primary key[R]. ANSI Database Committee (X3H2) Working Paper X3H2-84-118, 1984.

本章知识点讲解微视频:



关系模型



关系代数



关系演算

除法运算

第3章

关系数据库标准语言 SQL

结构化查询语言(SQL)是关系数据库的标准语言,包括数据查询、数据库模式创建、数据库数据的增删改、数据库安全性和完整性定义与控制等一系列功能。

本章导读

本章详细介绍 SQL 的基本功能,并进一步讲述关系数据库的基本概念,关系数据库管理系统(RDBMS)对数据库三级模式的支持。

本章首先讲述了 SQL 的产生与发展、SQL 的特点,非过程化 SQL 和过程化语言的区别。读者可以从中进一步了解关系数据库技术和关系数据库管理系统产品的发展过程,从而体会关系数据库系统能够减轻用户负担,提高用户开发数据库应用系统生产率的优点。然后通过实例详细讲解 SQL 的功能、语法和使用要点。

SQL 功能丰富、简洁易懂,不过根据需求正确无误地写出 SQL 语句并不容易,特别是使用 SQL 完成复杂查询更需要认真练习才能掌握。

希望读者通过上机实际操作掌握 SQL 的运用,包括数据表定义、数据的查询、插入、删除、更新,以及索引和视图的创建和删除等操作。

本章的难点是正确写出 SQL 语句并完成各种查询,学习方法就是反复训练,通过具体的数据库产品上进行实际练习,才能举一反三牢固掌握。

3.1 SQL 概述

自 SQL 成为数据库国际标准语言以来,几乎所有数据库厂商都采用了 SQL,而且许多软件厂商也纷纷推出与 SQL 接口的软件,使不同数据库系统之间、数据库系统与其他软件系统之间的互操作有了共同的基础。其意义重大,因此有人把确立 SQL 为关系数据库标准语言及其后的发展称为是一场革命。

3.1.1 SQL 的产生与发展

SQL 是在 1974 年由 Boyce 和 Chamberlin 提出的,最初的名称是 SEQUEL(structured English